

Architectural Analysis and Remediation Strategy for the BreakingFlashcards Video Processing Pipeline

I. Deconstructing the SIGABRT Failure: A Categorical Analysis of Concurrency Conflicts

1.1. Initial Diagnosis: Interpreting the EXC_CRASH (SIGABRT) Signal

The primary diagnostic artifact is the crash report indicating the application, BreakingFlashcards, terminated on Thread 1: com.apple.main-thread (serial) with a signal SIGABRT. The assembly-level view shows the crash occurring within the libsystem_kernel.dylib library, specifically at the `__pthread_kill` function call. This information is crucial for an accurate diagnosis.

A SIGABRT signal, corresponding to the exception code EXC_CRASH, is not an arbitrary or accidental memory access error. It represents a "signal abort," a deliberate termination initiated by the process itself.¹ This typically occurs when the application's runtime environment detects an unrecoverable internal state, most often as the result of an unhandled language exception (e.g., in Swift or Objective-C) or a failed assertion.³ The

`__pthread_kill` function is the low-level mechanism that delivers this termination signal to the specified thread; it is the final step in the termination process, not the source of the error.³ The true fault lies earlier in the execution path, where an exceptional condition was raised but not caught.

The user reports that these crashes occur intermittently during the "saving of the move stage." This application phase involves complex, resource-intensive operations, including video transcoding, file I/O, and database transactions with CoreData. Executing such

long-running tasks on the main thread is a known anti-pattern that can lead to application hangs or, more critically in this case, create the conditions for severe concurrency errors.

1.2. The Prime Suspect: CoreData Concurrency Violation

Given the operational context—video processing followed by a save operation—the most probable cause of the unhandled exception is a CoreData concurrency violation. CoreData's architecture enforces a strict thread confinement model: an instance of `NSManagedObjectContext` and any `NSManagedObjects` fetched from it are bound to the specific Grand Central Dispatch (GCD) queue on which they were created. Accessing these objects from any other queue is a programmer error that leads to data corruption, unpredictable behavior, and often, a crash.⁵

The likely sequence of events leading to the observed SIGABRT is as follows:

1. A video processing task (e.g., trimming via `AVAssetExportSession`) is initiated, running asynchronously on a background queue to avoid blocking the user interface.
2. Upon completion of this background task, its completion handler attempts to save the results. This involves creating or updating a `MoveEntity` managed object.
3. The completion handler, still executing on the background queue, improperly accesses the application's main `NSManagedObjectContext`, which is confined to the main thread.
4. The CoreData framework detects this illegal cross-thread access and raises a fatal exception to prevent data corruption.⁷
5. Because this exception is unhandled by the application code, the runtime's default exception handler is invoked, which calls the `abort()` function, leading to the SIGABRT signal being sent to the process.³

The crash is observed on the main thread because it is the application's primary execution context where the runtime often finalizes the termination process. However, the fault originates in the logic of the background task that initiated the illegal state mutation. This distinction is paramount for effective debugging; focusing exclusively on the main thread's state at the moment of the crash is misleading. The investigation must trace the flow of data and control from the background video processing task to the CoreData persistence layer.

From a categorical perspective, this failure represents an invalid composition of morphisms (functions). Let the main thread's `NSManagedObjectContext` be an object $SC_{\{main\}}$. Let $f_{\{export\}}: VideoAsset \rightarrow FileURL$ be the video export morphism executing on a background thread. Let $g_{\{save\}}: FileURL \rightarrow ()$ be the CoreData save morphism. The current implementation incorrectly attempts the composition $g_{\{save\}}(f_{\{export\}}(asset))$, where $g_{\{save\}}$ internally and illegally accesses $SC_{\{main\}}$. The composition is invalid because the morphism $g_{\{save\}}$ violates the thread-confinement

rules of its target object, `$C_{main}$`.

1.3. Immediate Diagnostic Action: Enabling Concurrency Debugging

To confirm this hypothesis and pinpoint the exact location of the error, the most effective first step is to enable CoreData's concurrency debugging features. By adding the launch argument `-com.apple.CoreData.ConcurrencyDebug 1` to the application's scheme in Xcode, the CoreData framework is placed into a heightened state of vigilance.⁸ In this mode, any violation of the thread confinement rules will cause an immediate and deterministic crash at the precise line of code where the illegal access occurs, rather than later at an unpredictable point in the program's execution.⁷ This tool will transform the intermittent, difficult-to-reproduce bug into a consistent failure, providing an unambiguous starting point for refactoring.

Crash Analysis Summary			
Symptom	Observed in Log	Hypothesized Root Cause	Supporting Evidence
App abruptly crashes during the saving stage of the "add move" flow.	Thread 1: signal SIGABRT delivered via <code>__pthread_kill</code> .	A background thread (likely from a video export completion handler) is attempting to access and mutate an <code>NSManagedObjectContext</code> that is confined to the main thread.	³
Failures are intermittent and not 100% reproducible under normal	The stack trace does not immediately point to application code,	The crash is caused by a race condition. The timing of thread execution	¹¹

conditions.	ending in system libraries.	determines whether the illegal access occurs, making the bug appear random.	
-------------	-----------------------------	---	--

II. Modeling the "Add Move" Flow: Objects, Morphisms, and Compositional Breakdown

To understand the systemic nature of the failures, it is instructive to model the "add move" workflow using the language of category theory. In this model, data structures and system states are **objects**, and the functions or user actions that transition the system between these states are **morphisms**. The entire end-to-end flow is a **composition** of these morphisms. The observed instability arises from flaws in how these morphisms are composed, particularly across asynchronous boundaries.

2.1. Defining the System's Category

Objects (States and Data Structures):

- `$O_{Picker}$`: The initial state of the system, where no video has been selected by the user.
- `$O_{VideoAsset}$`: An `AVAsset` object, representing the immutable media asset chosen from the user's photo library.
- `$O_{TrimConfig}$`: A value type (e.g., a Swift struct) encapsulating the user-defined trim and transform parameters, such as start/end `CMTime` and a `CGAffineTransform` for rotation.
- `$O_{Exporting}$`: A transient state representing the in-progress video export operation. This object would hold a reference to the active `AVAssetExportSession` and its progress.
- `$O_{ExportedVideo}$`: A URL object pointing to the temporary location on disk of the successfully processed, trimmed, and rotated video file.
- `$O_{MoveEntity}$`: An `NSManagedObject` instance representing the final, persisted "move" record within the `CoreData` database.
- `$O_{PhotoKitAsset}$`: A `PHAsset` object representing the video after it has been saved to the user's "BreakDex" photo album.
- `$O_{UIState}$`: The collective state of the user interface, encompassing the visibility of

loading indicators, the content of the video preview, and the enabled status of buttons.

Morphisms (Functions and State Transitions):

- $m_{\text{select}}: O_{\text{Picker}} \rightarrow O_{\text{VideoAsset}}\$$: The user action of selecting a video from the media library.
- $m_{\text{configureTrim}}: (O_{\text{VideoAsset}}, \text{UserInput}) \rightarrow O_{\text{TrimConfig}}\$$: The user interaction of adjusting the trim handles and rotation controls, resulting in a configuration object.
- $m_{\text{export}}: (O_{\text{VideoAsset}}, O_{\text{TrimConfig}}) \rightarrow O_{\text{Exporting}}\$$: The function call that initiates the AVAssetExportSession, transitioning the system into an exporting state. This is an asynchronous operation.
- $m_{\text{exportComplete}}: O_{\text{Exporting}} \rightarrow O_{\text{ExportedVideo}}\$$: The successful completion of the export session, yielding a URL to the final video file.
- $m_{\text{saveCoreData}}: (O_{\text{ExportedVideo}}, \text{MoveName}) \rightarrow O_{\text{MoveEntity}}\$$: The function that persists the move's metadata and a reference to the video file into CoreData.
- $m_{\text{savePhotoKit}}: O_{\text{ExportedVideo}} \rightarrow O_{\text{PhotoKitAsset}}\$$: The function that saves the video file to the designated album in the user's Photos library.
- $m_{\text{updateUI}}: (\text{AnyState}) \rightarrow O_{\text{UIState}}\$$: A general morphism representing the update of the UI to reflect the current state of the system.

2.2. The Flawed Composition

The end-to-end user flow can be described as the following composition of morphisms:

$m_{\text{savePhotoKit}} \circ m_{\text{saveCoreData}} \circ m_{\text{exportComplete}} \circ m_{\text{export}} \circ m_{\text{configureTrim}} \circ m_{\text{select}}\$$

The application's instability stems from the fact that this composition is not managed atomically or with regard for the execution context (i.e., thread) of each morphism. The control flow is likely imperative and distributed across various callbacks and completion handlers, leading to several points of breakdown.

- **Breakdown Point 1 (Preview Failure):** The user reports that "sometimes the preview fails to display in the NameMoveView." This suggests that the composition of $m_{\text{updateUI}} \circ m_{\text{configureTrim}}\$$ is flawed. Generating a preview from a trimmed AVAsset can be a non-trivial operation. If this is performed synchronously on the main thread, it can cause the UI to become unresponsive. If performed asynchronously, a race condition can occur where the UI attempts to update before the preview is ready, or the state management is not robust enough to handle the asynchronous result.
- **Breakdown Point 2 (Save Failure):** This is the critical failure point identified in Section I. The composition $m_{\text{saveCoreData}} \circ m_{\text{exportComplete}}\$$ is broken. The

morphism `$m_{exportComplete}$` concludes on a background thread, but the subsequent morphism, `$m_{saveCoreData}$`, is implemented in a manner that illegally interacts with main-thread-only CoreData objects. This is a classic race condition where the correctness of the program depends on the unpredictable timing of thread execution.¹¹

The architectural root of this problem is the absence of a dedicated state coordinator object. The system's fragility is a direct consequence of lacking a central entity responsible for managing these state transitions and ensuring the integrity of morphism compositions. Without a "Coordinator" or "State Machine" pattern, each component (e.g., a view controller or a service) is left to imperatively trigger the next step in the chain, often within a simple completion block. This distributed control flow is notoriously difficult to manage in a concurrent environment, making it nearly impossible to enforce crucial invariants like thread safety, error propagation, and atomicity across the entire end-to-end process. There is no single source of truth to answer the question, "What is the exact state of the 'add move' flow right now?" A dedicated coordinator object would encapsulate the state and the valid transitions between states, ensuring that morphisms like `$m_{saveCoreData}$` are always composed correctly and executed on the appropriate queue.

III. Architectural Remediation for the Video Processing Pipeline

The video processing pipeline, managed by AVFoundation, is a resource-intensive component of the "add move" flow. Its implementation must be robust, efficient, and non-interfering with the application's UI responsiveness.

3.1. Offloading Intensive Work: Background Export

Video trimming and exporting are computationally expensive operations that involve decoding, transforming, and re-encoding media data. `AVAssetExportSession` is the appropriate high-level API for these tasks.¹⁴ Executing an export session on the main thread is a critical performance error. It will block the UI, leading to a frozen interface, and if the operation takes too long, the application will be terminated by the operating system's watchdog timer.¹⁶

The definitive solution is to dispatch all `AVAssetExportSession` operations to a dedicated

background queue. Using a serial queue is highly recommended. This ensures that only one export operation occurs at a time, preventing multiple concurrent exports from overwhelming the device's CPU and memory resources. This is particularly important for maintaining stability on older devices with more limited hardware capabilities.¹⁸

A robust implementation should encapsulate this logic within a dedicated service class:

Swift

```
import AVFoundation
```

```
// In a dedicated VideoProcessingService class
```

```
private let exportQueue = DispatchQueue(label: "com.breakdex.video-export-queue")
```

```
func export(asset: AVAsset, trimConfig: TrimConfig, completion: @escaping (Result<URL, Error>) -> Void) {
```

```
    // Ensure this function is called from the main thread to initiate, but the work is done in the background.
```

```
    let temporaryURL = //... create a unique temporary file URL...
```

```
    guard let exportSession = AVAssetExportSession(asset: asset, presetName: AVAssetExportPresetHighestQuality) else {
        completion(.failure(ExportError.sessionCreationFailed))
        return
    }
```

```
    exportSession.outputURL = temporaryURL
    exportSession.outputFileType = .mp4
    exportSession.timeRange = trimConfig.timeRange
    exportSession.videoComposition = //... create AVMutableVideoComposition for rotation...
    exportSession.shouldOptimizeForNetworkUse = true
```

```
    exportQueue.async {
        exportSession.exportAsynchronously {
            DispatchQueue.main.async {
                switch exportSession.status {
                case.completed:
                    completion(.success(temporaryURL))
                case.failed:
```

```

        completion(.failure(exportSession.error ?? ExportError.unknown))
    case.cancelled:
        completion(.failure(ExportError.cancelled))
    default:
        completion(.failure(ExportError.unknown))
    }
}
}
}
}
}

```

3.2. Memory Management for Large Video Files

Processing large video files, especially at high resolutions like 4K, can exert significant memory pressure on the application. If memory usage exceeds system-defined limits, the OS will terminate the app with an `EXC_RESOURCE` exception.³ While

AVFoundation is engineered to handle large media files efficiently through streaming and hardware acceleration, developers must still adhere to best practices to avoid memory-related issues.¹⁹

The primary risk is holding large amounts of decoded video data in memory simultaneously. `AVAssetExportSession` mitigates this by processing the video as a stream, but the overall workflow must be managed with care.

- **Asset Initialization:** When creating the initial `AVURLAsset`, the `AVURLAssetPreferPreciseDurationAndTimingKey` option should be set to `true`. This ensures that the trim times are frame-accurate, which is essential for a WYSIWYG experience, even though it may incur a small initial performance cost.¹⁴
- **Temporary File Cleanup:** The `AVAssetExportSession` writes its output to a temporary file. Once this file has been successfully persisted (i.e., copied to a permanent location in the app's sandbox and saved to the PhotoKit library), it is imperative that this temporary file is deleted immediately. Failing to do so will cause the app's storage footprint to grow indefinitely, eventually leading to poor performance or OS-level cleanup.

It is important to recognize that high memory pressure can have secondary effects that increase system instability. When an application's memory usage is high, the operating system's scheduler may behave differently, altering the timing of thread execution in unpredictable ways. This can significantly increase the probability that a latent race condition or threading violation will manifest as a crash. A system that appears stable when processing

small, 10-second videos may fail consistently with 2-minute 4K videos, not just due to hitting a hard memory limit, but because the extended processing time and heightened resource contention create a more hostile environment for buggy concurrent code. Therefore, diligent memory management is not only about preventing memory-specific crashes but is also a crucial component of ensuring overall concurrent stability.

IV. A Resilient Persistence Layer: Correcting CoreData and PhotoKit Morphisms

The persistence layer is responsible for the final, critical steps of the "add move" flow. Its implementation must be both thread-safe and idempotent to ensure data integrity and a reliable user experience.

4.1. CoreData: Thread Safety and Data Storage Strategy

The SIGABRT crash strongly points to flaws in the CoreData implementation. Correcting this requires addressing both the storage strategy for video data and the mechanism for performing writes from a background thread.

Storage Strategy: Storing large binary objects (BLOBs), such as video files, directly within a CoreData SQLite store is a significant anti-pattern. It dramatically increases the size of the database file, which can degrade the performance of fetches, saves, and migrations.²¹ While CoreData offers an "Allows External Storage" option that transparently stores large attributes on the file system²², a more explicit and robust approach is to manage the files manually. The recommended strategy is to save the processed video file to a permanent, private location within the application's sandbox (e.g., the

Application Support directory) and store only the relative file path as a String attribute on the MoveEntity managed object.²³ This keeps the database lightweight and performant.

Thread-Safe Saving: All write operations to CoreData originating from a background task must be performed on a dedicated background context. The NSPersistentContainer API provides the `performBackgroundTask(_:)` method, which is the modern, preferred way to handle this. This method yields a temporary, private queue `NSManagedObjectContext` that is properly configured for safe background processing.⁵ To communicate the result of the save

back to the main thread (e.g., for a UI update), the thread-safe

NSManagedObjectID must be used, as passing the NSManagedObject instance itself across thread boundaries is forbidden.⁶

The following code demonstrates a correct, thread-safe save operation:

Swift

```
import CoreData
```

```
// In a dedicated CoreDataPersistenceService class
```

```
func saveNewMove(name: String, finalVideoURL: URL, completion: @escaping  
(Result<NSManagedObjectID, Error>) -> Void) {
```

```
    let container = persistentContainer // Reference to your app's NSPersistentContainer
```

```
    container.performBackgroundTask { backgroundContext in
```

```
        // backgroundContext is a new, temporary context on a private queue.
```

```
        backgroundContext.mergePolicy = NSMergeByPropertyObjectTrumpMergePolicy
```

```
        let newMove = MoveEntity(context: backgroundContext)
```

```
        newMove.name = name
```

```
        newMove.creationDate = Date()
```

```
        // Assume saveVideoToPermanentStorage copies the file and returns its relative path
```

```
        do {
```

```
            let relativePath = try self.saveVideoToPermanentStorage(from: finalVideoURL)
```

```
            newMove.videoPath = relativePath
```

```
            try backgroundContext.save()
```

```
            // Safely pass the result back to the main thread using the thread-safe objectID
```

```
            let objectID = newMove.objectID
```

```
            DispatchQueue.main.async {
```

```
                completion(.success(objectID))
```

```
            }
```

```
        } catch {
```

```
            DispatchQueue.main.async {
```

```
                completion(.failure(error))
```

```
            }
```

```

    }
  }
}

```

4.2. PhotoKit: Idempotent Album Management and Asset Saving

The requirement to save videos to a specific "BreakDex" album, creating it only if it doesn't already exist, necessitates careful interaction with the PhotoKit framework. All modifications to the user's photo library must be wrapped in a `PHPhotoLibrary.shared().performChanges(_:completionHandler:)` block, which handles user permissions and ensures atomic updates.²⁷

The process must be idempotent—that is, running it multiple times should produce the same result as running it once, without creating duplicate albums. This is achieved by first fetching asset collections to check for an existing album with the correct title. Only if the fetch result is empty should a creation request be made.²⁹

The following helper functions encapsulate this logic:

Swift

```
import Photos
```

```
// In a dedicated PhotoKitService class
```

```
func findOrCreateAlbum(named albumName: String, completion: @escaping
(Result<PHAssetCollection, Error>) -> Void) {
```

```
    let fetchOptions = PHFetchOptions()
```

```
    fetchOptions.predicate = NSPredicate(format: "title = %@", albumName)
```

```
    let collections = PHAssetCollection.fetchAssetCollections(with:.album, subtype:.any, options:
fetchOptions)
```

```
    if let album = collections.firstObject {
```

```
        // Album already exists
```

```
        completion(.success(album))
```

```
        return
```

```
    }
```

```

// Album does not exist, create it
var albumPlaceholder: PHObjectPlaceholder?
PHTPhotoLibrary.shared().performChanges({
    let createAlbumRequest =
PHAssetCollectionChangeRequest.creationRequestForAssetCollection(withTitle: albumName)
    albumPlaceholder = createAlbumRequest.placeholderForCreatedAssetCollection
}) { success, error in
    if success, let placeholder = albumPlaceholder {
        let newlyCreatedAlbum = PHAssetCollection.fetchAssetCollections(withLocalIdentifiers:
[placeholder.localIdentifier], options: nil)
        if let album = newlyCreatedAlbum.firstObject {
            completion(.success(album))
        } else {
            completion(.failure(PhotoKitError.albumCreationFailed))
        }
    } else {
        completion(.failure(error ?? PhotoKitError.unknown))
    }
}
}

func saveVideo(from url: URL, to album: PHAssetCollection, completion: @escaping (Result<Void,
Error>) -> Void) {
    PHTPhotoLibrary.shared().performChanges({
        guard let assetRequest =
PHAssetChangeRequest.creationRequestForAssetFromVideo(atFileURL: url),
        let placeholder = assetRequest.placeholderForCreatedAsset,
        let albumChangeRequest = PHAssetCollectionChangeRequest(for: album)
    else { return }

        albumChangeRequest.addAssets([placeholder] as NSFastEnumeration)
    }) { success, error in
        if success {
            completion(.success(()))
        } else {
            completion(.failure(error ?? PhotoKitError.unknown))
        }
    }
}
}

```

A critical causal relationship must be enforced: the PhotoKit save operation is dependent on the successful completion of both the AVAssetExportSession and the subsequent move of the temporary video file to its permanent location in the app's sandbox. The control flow must

ensure that a failure at any preceding step prevents the PhotoKit save from being attempted. This again highlights the architectural need for a state coordinator to correctly manage the dependency graph of these asynchronous morphisms.

V. Enhancing User Perception: Asynchronous Operations and Progress Indication

The user experience during long-running, asynchronous tasks is a critical component of application quality. The current implementation suffers from two key UX flaws: a distracting loading indicator during trimming and an uninformative progress indicator for longer uploads.

5.1. The Trimming Loading Indicator: Eliminating Visual Noise

The user describes the loading indicator shown after the trimming view as "too visually distracting" and "colorful." Furthermore, for very short operations, a loading indicator that appears and disappears almost instantly creates a "flicker" effect, which users perceive as visual noise or a glitch.³¹

The recommendation is twofold:

1. **Change the Style:** Replace the custom, colorful indicator with a standard, minimal `UIActivityIndicatorView` (UIKit) or a `ProgressView` in its indeterminate, circular style (SwiftUI).³² These system-standard components are less intrusive and more familiar to users.
2. **Introduce a Delay:** To eliminate the flicker effect for fast operations, the appearance of the loading indicator should be delayed. It should only become visible if the operation takes longer than a brief, perceptible threshold, such as 0.5 seconds.³¹ If the operation completes within this delay period, the indicator is never shown, resulting in a cleaner and more responsive-feeling interface.

5.2. The Upload Progress Indicator: Managing User Expectations

For longer video processing tasks, the user reports that the progress indicator gets "stuck on

10%." This is a common and frustrating user experience issue. Human interface guidelines stress that progress indicators should, whenever possible, be determinate and should be kept in motion to reassure the user that the application has not stalled.³⁴ A static progress bar erodes user trust and can lead them to abandon the task or force-quit the application.

The "stuck at 10%" problem often occurs when progress is not linear. The underlying task—in this case, `AVAssetExportSession`—may have an initial setup phase that completes quickly, followed by a long, computationally intensive processing phase with infrequent progress updates.

The recommended solution is to provide more honest and multi-faceted feedback:

1. **Use Determinate Progress:** The progress property of an `AVAssetExportSession` is a Float value between 0.0 and 1.0 that can be observed via Key-Value Observing (KVO) or a Timer. This value should be directly bound to the progress property of a `UIProgressView` (UIKit) or the value of a determinate `ProgressView` (SwiftUI).¹⁵
2. **Display Elapsed Time:** Since the percentage can be non-linear and slow to update, it should be supplemented with a simple elapsed time counter. When the export task begins, a Timer should be initiated to update a UI label every second, displaying the time since the task began (e.g., "Processing... 0:15"). This provides constant, moving feedback, reassuring the user that the process is active even when the progress bar itself is not advancing.³⁶

This dual-feedback approach builds user trust by communicating more honestly about the state of the task. It acknowledges that the process is ongoing (via the elapsed time) while providing the best available estimate of completion (via the progress bar). This manages user expectations far more effectively than a single, potentially misleading metric.

VI. Strategic Roadmap for Implementation and Validation

The following is a prioritized, phased roadmap to guide the remediation of the identified issues. This sequence is designed to address the most critical stability issues first, followed by correctness and user experience enhancements.

6.1. Phase 1: Stabilize and Diagnose (Priority 1)

This phase focuses on confirming the root cause of the crash and making the system's failure modes predictable.

- **Action 1.1:** Enable CoreData Concurrency Debugger. In the Xcode project's scheme editor, under "Run" -> "Arguments", add the launch argument `-com.apple.CoreData.ConcurrencyDebug 1`.⁸
- **Action 1.2:** Reproduce the Crash. With the debugger attached, perform the "add move" flow with a video. The application is now expected to crash deterministically at the exact line of code where the cross-thread CoreData access occurs. This action validates the primary hypothesis from Section I.
- **Action 1.3:** Implement an Exception Breakpoint. In Xcode's Breakpoint Navigator, add a global "Exception Breakpoint." This will pause the debugger at the moment an exception is thrown, often providing a more complete stack trace than the final crash report.¹

6.2. Phase 2: Refactor the Pipeline for Correctness (Priority 2)

This phase involves re-architecting the video processing and persistence layers according to best practices.

- **Action 2.1:** Encapsulate video processing logic in a dedicated service class. Introduce a serial DispatchQueue for all AVAssetExportSession operations.
- **Action 2.2:** Refactor the CoreData save logic to use `persistentContainer.performBackgroundTask`. Ensure all NSManagedObject creation and modification occurs within the provided background context.
- **Action 2.3:** Modify the MoveEntity in the CoreData model to store the video's location as a String path instead of Binary Data. Implement the file management logic to copy the exported video to a permanent location within the app's sandbox and delete the temporary source file.
- **Action 2.4:** Implement the thread-safe handoff pattern. The background persistence task should return the NSManagedObjectID of the newly created MoveEntity to the main thread for any required UI updates (e.g., navigating to the detail view).
- **Action 2.5:** Encapsulate PhotoKit logic in a service class. Implement the idempotent `findOrCreateAlbum` function and ensure the video saving logic is robust and handles all completion and error states correctly.

6.3. Phase 3: Enhance User Experience (Priority 3)

This final phase addresses the user-facing issues with loading and progress indicators.

- **Action 3.1:** Replace the current trimming loading indicator with a standard, minimal `UIActivityIndicatorView` or `ProgressView`. Implement a 0.5-second delay before it appears to prevent UI flicker on fast operations.
- **Action 3.2:** For the longer export process, implement a determinate `UIProgressView`. Bind its progress to the progress property of the `AVAssetExportSession`.
- **Action 3.3:** Add a `UILabel` adjacent to the progress view. Implement a `Timer` that starts when the export begins and stops on completion, updating the label with the elapsed time in a MM:SS format.

Recommended Architectural Changes				
Component	Current Flaw	Recommended Change	Primary Benefit	Priority
CoreData Persistence	Cross-thread <code>NSManagedObjectContext</code> access causes SIGABRT crashes.	Use <code>persistentContainer.performBackgroundTask</code> for all write operations originating from background threads.	Crash Elimination	1
CoreData Schema	Storing large video binary data in the database bloats the store and degrades performance.	Store videos as files in the app's sandbox; store only the file path String in CoreData.	Performance & Scalability	2
AVFoundation Export	Export operations	Execute <code>AVAssetExport</code>	UI Responsiveness	2

	block the main thread, causing UI freezes and potential watchdog terminations.	Session on a dedicated background serial DispatchQueue.	ss	
PhotoKit Integration	Potential for creating duplicate "BreakDex" albums; no robust error handling.	Implement an idempotent "find-or-create" album logic before saving the video asset.	Data Integrity	2
UI/UX (Trimming)	A visually distracting loading indicator flickers on screen for short operations.	Replace with a minimal, system-standard indicator that appears only after a 0.5s delay.	Improved User Experience	3
UI/UX (Export)	Progress indicator stalls for long operations, providing poor feedback and eroding user trust.	Display both a percentage from AVAssetExportSession.progress and a constantly updating elapsed time counter.	Improved User Trust	3

Works cited

1. How To Solve SIGABRT Error in Xcode - Appy Pie Vibe, accessed September 26, 2025, <https://www.appypievibe.ai/blog/swift-code/sigabrt-xcode-swift/>
2. How to solve a Thread 1: signal SIGABRT error? [closed] - Stack Overflow,

accessed September 26, 2025,

<https://stackoverflow.com/questions/59887252/how-to-solve-a-thread-1-signal-sigabrt-error>

3. EXC_CRASH (SIGABRT) | Apple Developer Documentation, accessed September 26, 2025, <https://developer.apple.com/documentation/xcode/sigabrt>
4. Resolving SIGABRT Issues in iOS App | by Deepak Carpenter ..., accessed September 26, 2025, <https://medium.com/appgrid/resolving-sigabrt-issues-in-ios-app-ec634d8bad72>
5. Concurrency - Core Data Programming Guide - Apple Developer, accessed September 26, 2025, <https://developer.apple.com/library/archive/documentation/Cocoa/Conceptual/CoreData/Concurrency.html>
6. Using Core Data in the background | Apple Developer Documentation, accessed September 26, 2025, <https://developer.apple.com/documentation/coredata/using-core-data-in-the-background>
7. ios - Swift crashing with signal SIGABRT with the core data - Stack ..., accessed September 26, 2025, <https://stackoverflow.com/questions/73793379/swift-crashing-with-signal-sigabrt-with-the-core-data>
8. Making com.apple.CoreData.ConcurrencyDebug 1 work - Stack Overflow, accessed September 26, 2025, <https://stackoverflow.com/questions/31391838/making-com-apple-coredata-concurrencydebug-1-work>
9. Using launch arguments for easier Core Data and SwiftData debugging - Donny Wals, accessed September 26, 2025, <https://www.donnywals.com/using-launch-arguments-for-easier-core-data-debugging/>
10. Is this a valid way of debugging CoreData concurrency issues? - Stack Overflow, accessed September 26, 2025, <https://stackoverflow.com/questions/41176098/is-this-a-valid-way-of-debugging-coredata-concurrency-issues>
11. Understanding and Managing Race Conditions in Swift | by Essam Fahmy - Medium, accessed September 26, 2025, <https://medium.com/@essammohamedomran/understanding-and-managing-race-conditions-in-swift-43e128a12800>
12. Avoiding race conditions in Swift - Swift by Sundell, accessed September 26, 2025, <https://www.swiftbysundell.com/articles/avoiding-race-conditions-in-swift/>
13. Handling Race Conditions in iOS with DispatchSemaphore - Osas Blogs, accessed September 26, 2025, <https://swiftlogic.io/posts/handling-race-conditions-in-ios/>
14. Trim video using AVFoundation in Swift - GitHub Gist, accessed September 26, 2025, <https://gist.github.com/b8c5f8eafe0605a38692>
15. How to Use AVFoundation Framework to Trim the Video - Bacancy Technology, accessed September 26, 2025, <https://www.bacancytechnology.com/blog/avfoundation-framework-to-trim-the->

[video](#)

16. Identifying the cause of common crashes | Apple Developer Documentation, accessed September 26, 2025, <https://developer.apple.com/documentation/xcode/identifying-the-cause-of-common-crashes>
17. Choosing Background Strategies for Your App | Apple Developer Documentation, accessed September 26, 2025, <https://developer.apple.com/documentation/backgroundtasks/choosing-background-strategies-for-your-app>
18. Tracking Down Memory Leaks with Instruments - CueCam Devlog 2 #xcode #swift, accessed September 26, 2025, <https://www.youtube.com/watch?v=j8y-LtRV4hM>
19. AVFoundation Overview - Apple Developer, accessed September 26, 2025, <https://developer.apple.com/av-foundation/>
20. Unlocking AVFoundation in Swift: The Ultimate Developer's Guide | by Muhammed Yarbash K | Medium, accessed September 26, 2025, <https://medium.com/@muhammedyarbashk/the-media-magician-transforming-swift-apps-with-avfoundation-bff7600104d2>
21. How to Save Images and Videos to Core Data Efficiently - Yet Another Swift Blog, accessed September 26, 2025, <https://www.vadimbulavin.com/how-to-save-images-and-videos-to-core-data-efficiently/>
22. How can I save photos/videos to my app? (CoreData??) : r/iOSProgramming - Reddit, accessed September 26, 2025, https://www.reddit.com/r/iOSProgramming/comments/x58xbv/how_can_i_save_photos_videos_to_my_app_coredata/
23. How to save and retrieve .mp4 in Core Data (Swift) - Stack Overflow, accessed September 26, 2025, <https://stackoverflow.com/questions/32851576/how-to-save-and-retrieve-mp4-in-core-data-swift>
24. Storing photos and videos in Core Data? - Stack Overflow, accessed September 26, 2025, <https://stackoverflow.com/questions/4545527/storing-photos-and-videos-in-core-data>
25. Teaching Core Data to dance with threads | by Duncan Anderson - Medium, accessed September 26, 2025, <https://duncsand.medium.com/threading-43a9081284e5>
26. Three Common Core Data Mistakes to Avoid - Cocoacasts, accessed September 26, 2025, <https://cocoacasts.com/three-common-core-data-mistakes-to-avoid>
27. Requesting Changes to the Photo Library | Apple Developer Documentation, accessed September 26, 2025, <https://developer.apple.com/documentation/photokit/requesting-changes-to-the-photo-library>
28. Browsing and Modifying Photo Albums | Apple Developer Documentation, accessed September 26, 2025,

<https://developer.apple.com/documentation/photokit/browsing-and-modifying-photo-albums>

29. How to download & save the video in device storage in swift | by Yasir - Medium, accessed September 26, 2025, <https://mdcode2021.medium.com/how-to-download-save-the-video-in-device-storage-in-swift-a1129aecd736>
30. Save video to a custom Album using Photos Framework in iOS - Stack Overflow, accessed September 26, 2025, <https://stackoverflow.com/questions/44669178/save-video-to-a-custom-album-using-photos-framework-in-ios>
31. A Delayable Progress Indicator in SwiftUI - Livsy Code, accessed September 26, 2025, <https://livsycode.com/swiftui/a-delayable-progress-indicator-in-swiftui/>
32. UIActivityIndicatorView | Apple Developer Documentation, accessed September 26, 2025, <https://developer.apple.com/documentation/uikit/uiactivityindicatorview>
33. Creating Various Loading Views in SwiftUI | by Garejakirit - Medium, accessed September 26, 2025, <https://medium.com/@garejakirit/creating-various-loading-views-in-swiftui-76e31509cdee>
34. Progress indicators | Apple Developer Documentation, accessed September 26, 2025, <https://developer.apple.com/design/human-interface-guidelines/progress-indicators>
35. UIProgressView | Apple Developer Documentation, accessed September 26, 2025, <https://developer.apple.com/documentation/uikit/uiprogressview>
36. Progress | Apple Developer Documentation, accessed September 26, 2025, <https://developer.apple.com/documentation/foundation/progress>